Temporizadores de hardware

Un **temporizador** cuenta pulsos de reloj por sí solo. Cuando el contador llega a la **alarma**, avisa con una **interrupción** y corre una **ISR** corta que levanta una **bandera**.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Del reloj a la ISR

la CPU no cuenta: el timer cuenta solo

1 — Reloj base · 80 MHz `RELOJ`
80 000 000 pulsos por segundo. Cada pulso dura **12,5 ns**.

2 — Prescaler ÷ N `DIVISOR`
Deja pasar **1 de cada N** pulsos. Cada pulso que pasa es un **tick**.

```
f = 80 000 000 / N
```

3 — Contador `CONTADOR`
Suma **1 por tick**. Con N = 80, 1 tick = **1 µs**.

4 — contador = alarma

Sí → **Interrupción → ISR**
ISR: función corta que corre cuando llega el aviso.

No → No: el contador sigue contando (paso 3)

2 Qué fórmula usar

según el dato que busco

Qué busco

La alarma
Tengo el tiempo y f.

```
alarma = tiempo × f
```

El tiempo
Tengo la alarma y f.

```
tiempo = alarma / f
```

El prescaler
Tengo f.

```
N = 80 000 000 / f
```

LAS FÓRMULAS

FRECUENCIA DE TICK `f = 80 000 000 / N`	PRESCALER `N = 80 000 000 / f`	UN TICK DURA `1 / f`
TIEMPO HASTA EL AVISO `tiempo = alarma / f`	ALARMA PARA UN TIEMPO `alarma = tiempo × f`	

3 Ejemplo: un aviso cada 0,5 s

ticks de 1 MHz (f = 1 000 000)

1. **Prescaler** — N = 80 000 000 / 1 000 000 → `N = 80`
2. **Un tick dura** — 1 / 1 000 000 s → `1 µs`
3. **Alarma** — 0,5 × 1 000 000 → `500 000`
4. **Comprobación** — 500 000 × 1 µs → `0,5 s`

CON TICKS DE 1 MHZ

Cada tick dura 1 µs, así que la alarma se escribe directamente en **µs**.

4 Autorrecarga

el aviso se repite solo

1. **El contador cuenta**
   0, 1, 2, 3…
2. **Llegó a la alarma**
   - No → **Sigue contando**. Vuelve al paso 1.
   - Sí →
3. **Aviso**
   Se genera la interrupción.
4. **Vuelve a 0**
   Es la **autorrecarga** (*auto-reload*).

`vuelve al paso 1: avisos a intervalos iguales`

Autorrecarga	vs	Desbordamiento
`VUELVE A 0` en la alarma que **tú eliges**		`VUELVE A 0` en el **máximo** de bits
`USO` normal		`USO` casi nunca intencional

TAMAÑO DEL PRESCALER N

N pequeño	vs	N grande
`TICKS` cortos		`TICKS` largos
`EFECTO` más precisión; se llena antes		`EFECTO` cubre más tiempo; menos precisión

5 De la ISR al loop()

el trabajo lento se hace fuera de la ISR

1 — Llega la alarma `TIMER`

El timer avisa.

2 — Corre la ISR `ISR`

Es un **callback**: función que escribes tú y llama el timer. Lleva `IRAM_ATTR`: se guarda en la **IRAM** (RAM interna rápida), no en la flash.

```
void IRAM_ATTR onTimer()
```

3 — Sube la bandera `BANDERA`

La ISR solo pone `bandera = true` y termina.

```
volatile bool tick
```

4 — `loop()` la revisa `LOOP()`

La baja y hace el trabajo lento.

`VOLATILE`

Obliga a leer la variable **de la memoria cada vez**. Sin `volatile`, `loop()` puede no ver el cambio que hizo la ISR.

6 ESP32 vs. ATmega328P

ESP32

`TIMERS` 4, en paralelo
`CONTADOR` 64 bits
`RELOJ` 80 MHz
`PRESCALER` N de 2 a 65 535

vs

ATmega328P (Uno)

`TIMERS` 3
`CONTADOR` 8 o 16 bits
`RELOJ` 16 MHz
`PRESCALER` 1, 8, 64, 256 o 1024

CÓDIGO (CORE 3.X)

```
hw_timer_t *timer = nullptr;
volatile bool tick = false;
void IRAM_ATTR onTimer() { tick = true; }      // ISR: solo la bandera

void setup() {
  timer = timerBegin(1000000);                 // f = 1 MHz → 1 tick = 1 µs
  timerAttachInterrupt(timer, &onTimer);       // registra el callback
  timerAlarm(timer, 500000, true, 0);          // alarma 500 000 µs, autorrecarga
}
```